SCALING LAW FOR CATASTROPHIC FORGETTING VIA GRADIENT PRODUCTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Catastrophic forgetting occurs when models lose performance on previously learned tasks after acquiring new ones. Although larger models are empirically observed to forget less, the theoretical origin of this effect remains unclear. In this work, we analyze forgetting in simple linear and nonlinear teacher-student models and introduce a gradient-product proxy that closely tracks forgetting. This formulation allows us to decompose the phenomenon into its main components. We show that the orthogonality of the output heads is the necessary condition underlying the $1/d$ scaling law of forgetting, where d is the hidden dimension. Other factors, such as initialization, task similarity, network architecture, and activation functions, play a secondary role, modulating but not overturning the dominant scaling behavior. Our results provide a step toward a principled understanding of how model capacity mitigates forgetting and clarify the role of gradient interference in continual learning.

1 INTRODUCTION

Deep neural networks trained on tasks sequentially often overwrite earlier knowledge, a failure mode referred to as catastrophic forgetting (CF) McCloskey & Cohen (1989). Despite decades of research and a diverse set of mitigation algorithms, including regularization, replay, and projection, a simple and predictive principle is still lacking. In particular, it remains unclear when and why interference arises, and how it scales with model width and depth.

Mitigation strategies typically constrain updates on parameters important to past tasks, e.g., Elastic Weight Consolidation (EWC) Kirkpatrick et al. (2017) and Synaptic Intelligence (SI) Zenke et al. (2017) slow learning on sensitive weights; Memory-Aware Synapses estimates importance online; replay (e.g., iCaRL, GEM/A-GEM) preserves exemplars or gradients; and orthogonalization (OGD, OWM, GPM) projects new gradients away from subspaces tied to prior tasks. Although these methods have proven effective in practice, they provide heterogeneous explanations of why interference occurs, particularly as architectures and scales vary Wang et al. (2024).

Representation analyses indicate that deeper layers are disproportionately responsible for forgetting, and that mitigation methods often act by stabilizing these layers Ramasesh et al. (2020). Meanwhile, model width interacts with the learning regime: in the infinite-width limit, training dynamics linearize around initialization, and the Neural Tangent Kernel (NTK) framework has been applied to study forgetting Doan et al. (2021). However, discrepancies between this idealized regime and the behavior of finite-width networks reveal the limitations of such analyses Wenger et al. (2024). Task similarity has also been shown to influence CF, with the strongest interference observed for intermediate similarities Ramasesh et al. (2020) and in some cases for high similarities Hiratani (2024). Collectively, these findings highlight the absence of a unified scaling account that jointly captures the effects of width, depth, and task similarity.

This work takes a step toward such an account in a controlled teacher–student setting. We demonstrate that a gradient-product proxy, defined as the squared inner product of task gradients in the feature extractor, effectively captures CF across both linear and nonlinear models. Under mild isotropy (random output heads), the expected gradient angle follows a $1/d$ scaling. Depth primarily modulates this scaling by increasing the average overlap of gradient updates at deeper layers.

1.1 CONTRIBUTIONS

The main contributions of this work are as follows:

- We derive a simple gradient-product proxy that identifies the key factors underlying the $1/d$ scaling, namely initialization and the alignment of output heads.
- We show that head orthogonality is the primary determinant of interference, while initialization and architecture act as secondary modifiers.
- We provide rigorous empirical validation by repeating all experiments with five random seeds, and by evaluating across linear models, shallow (1-layer) and deeper Multilayer Perceptrons (MLPs), as well as on split and permuted MNIST tasks Deng (2012) with varying initialization and task similarity.

1.2 SCOPE AND LIMITATIONS

This study focuses on teacher–student classification with MLPs. Although the proxy and datasets are relatively simple, the underlying geometric mechanism is generic and connects directly to projection-based continual learning methods. Extending the analysis to convolutional and transformer architectures, as well as to long task streams, represents an important direction for future work Farajtabar et al. (2019).

2 EXPERIMENTAL SETUP

In this section, we briefly describe our experimental setup. For more details, consult Appendix A.1. Table 1 summarizes the main notation used throughout the paper.

Symbol	Description
Θ	feature extractor
ϑ	output head
d	width, hidden dimension
t	task index

Table 1: Notation

We study catastrophic forgetting in a teacher-student learning framework using the MNIST-split dataset, where digits are partitioned into two disjoint tasks: Task t (e.g., digits 0–4) and Task t' (e.g., digits 5–9).

Each model consists of a feature extractor Θ and an output head ϑ_t, forming the predictor

$$f(X_t) = \text{feature_extractor}(X_t)\,\vartheta_t,$$

where $X_t \in \mathbb{R}^{1\times m}$ is a single input sample.

All the implemented feature extractors are summarized in Table 2.

Model	Operation	Parameters
Linear	$X_t\Theta_1$	$\Theta_1 \in \mathbb{R}^{m\times d}$
1-Layer MLP	$\sigma(X_t\Theta_1)$	$\Theta_1 \in \mathbb{R}^{m\times d}$
2-Layer MLP	$\sigma(\sigma(X_t\Theta_1)\Theta_2)$	$\Theta_1 \in \mathbb{R}^{m\times d},\ \Theta_2 \in \mathbb{R}^{d\times d}$
3-Layer MLP	$\sigma(\sigma(\sigma(X_t\Theta_1)\Theta_2)\Theta_3)$	$\Theta_1 \in \mathbb{R}^{m\times d},\ \Theta_2 \in \mathbb{R}^{d\times d_2},\ \Theta_3 \in \mathbb{R}^{d_2\times d}$

Table 2: Implemented feature extractors. Here σ denotes a non-linear activation function.

Here, m is the size of a single data point $X_t \in \mathbb{R}^{1\times m}$, and d and d_2 are hidden dimensions, which are the main objects of our study.

The parameters are initialized $\sim \mathcal{U}(-a, a)$, where a can be chosen to be

- **capacity-dependent**, through Kaiming/He $a = \sqrt{\frac{3}{d}}$ He et al. (2015). This is a standard initialization strategy, but it introduces a scaling with model capacity.
- **constant** to remove any possible $1/d$ scaling.

2.1 Forgetting Experiment

Our teacher–student framework consists of two stages. First, we train two specialized teacher models independently, each dedicated to a distinct task. Subsequently, we perform sequential training of a dual-head student model. During student training, the respective teacher provides supervision for its associated task. When switching between tasks, the student activates the corresponding output head while maintaining a shared feature extractor. This is the typical structure of all forgetting experiments we carried out for this study.

2.2 Focused Gradient Updates

A critical constraint is that **all output heads are frozen during training** (`requires_grad = False`). This enforces the learning of task-specific representations exclusively through the shared feature extractor.

2.3 Training

Optimization is carried out using stochastic gradient descent (SGD) Ruder (2016) with a learning rate of $\eta = 0.01$. All models are trained with a batch size of 64 and through cross-entropy loss. Each teacher is trained for 50 epochs on its respective task. The student is then trained sequentially: first for 50 epochs on Task 1 under the supervision of Teacher 1, and subsequently for 50 epochs on Task 2 under the supervision of Teacher 2.

3 Analytical Model

To investigate the origin of the $1/d$ scaling, we begin with a simple analytical formulation. Consider two tasks X_t and $X_{t'}$, and a linear model of the form

$$f_{\text{linear}}(X_t) = X_t \Theta \, \vartheta_t, \tag{1}$$

where $X_t \in \mathbb{R}^{1\times m}$ is the input, $\Theta \in \mathbb{R}^{m\times d}$ is the feature extractor, and $\vartheta_t \in \mathbb{R}^{d\times o}$ is the output head. Gradient interactions, also referred to as gradient collision, have been proposed as a mechanism underlying CF Lopez-Paz & Ranzato (2022).

We define the proxy G for forgetting as the Frobenius product between the gradients of two subsequent tasks:

$$\begin{aligned} G_{\text{linear}} := \langle \nabla_\Theta \mathcal{L}t, \nabla\Theta \mathcal{L}_{t'} \rangle_F &= K_{\text{linear}} \langle X_t, X_{t'} \rangle \langle \vartheta_t, \vartheta_{t'} \rangle \\ &= K_{\text{linear}} \langle X_t, X_{t'} \rangle \|\vartheta_t\| \, \|\vartheta_{t'}\| \cos(\phi_{t,t'}), \end{aligned}$$

where K_{linear} is a constant, $\cos(\phi_{t,t'})$ denotes the cosine similarity between ϑ_t and $\vartheta_{t'}$, and the dot product $\langle X_t, X_{t'} \rangle$ reflects task similarity. Since $\vartheta \sim \mathcal{U}(-a, a)$, the expectation of G_{linear} vanishes, and higher-order moments must be considered. Under Kaiming/He initialization He et al. (2015), both $\text{Var}[\cos(\phi_{t,t'})]$ and $\text{Var}[\vartheta_{t,t'}]$ scale as $1/d$. More derivation details are in the Appendix A.3.

For a one-layer MLP, with ReLU activation σ, the model is

$$f_{\text{1-L MLP}}(X_t) = \sigma(X_t \Theta) \, \vartheta_t. \tag{2}$$

The corresponding forgetting proxy becomes

$$G_{\text{1-L MLP}} := \langle \nabla_\Theta \mathcal{L}t, \nabla\Theta\mathcal{L}_{t'} \rangle_F = K_{\text{1-L MLP}} \langle X_t, X_{t'} \rangle \cdot \text{Tr}(\vartheta_t^\top D_t D_{t'}^\top \vartheta_{t'}), \tag{3}$$

where $D_{t,t'}$ are $d \times d$ diagonal matrices whose elements are binary random variables $g_i \in \{0, 1\}$ coming from the ReLU derivative. Since those binary matrices do not introduce additional $1/d$ scaling, we deduce that the $1/d$ scaling has its origin in the output heads for both linear and one-layer MLP.

It is possible to define an *averaged* version of the proxies defined above by dividing them by d:

$$\bar{G} = \frac{1}{d} G. \tag{4}$$

On the one hand, the operation automatically introduces a scaling $1/d$, which might cover other effects that we wish to observe. On the other hand, averaging might provide a closer description of forgetting. For these reasons, we analyze both G and $\bar{G}$.

We summarize the intuitions developed in this theoretical section as follows:

The $1/d$ scaling originates from the output heads ϑ. Two possible mechanisms are identified:

1. The variance of the cosine similarity between the output heads carries the well-known "curse of dimensionality", which makes random vectors orthogonal in high dimension.
2. The variance of the heads themselves scales as $1/d$ for Kaiming/He initialization.

We find experimental evidence that the first is the dominant source of global $1/d$ scaling, and the second exerts only a secondary impact on the scaling rate.

Moreover, from the toy models above, we expect secondary effects coming from

1. Task similarity scales G globally, with higher forgetting expected for more similar tasks.
2. The ReLU non-linearity for the one-layer MLP multiplies the output heads with binary gates, changing their overall distribution. In fact, this operation accumulates plenty of mass at zero, and the distribution becomes mixed.
3. The constant K differs across models because it depends on residual loss terms, leading to model-dependent global scaling.

4 Experiments & Results

We conducted a series of *forgetting experiments*, systematically varying the key quantities listed in Table 3.

Quantity	**Options**
Feature Extractor	Linear, 1-/2-/3-Layer MLP
Hidden Dimension d	10, 50, 100, 150, 200
Equal Head	True, False
Initialization	Kaiming, Uniform

Table 3: Experimental factors and their configurations.

4.1 Standard Configuration

We performed the forgetting experiment with the standard settings (the parameters in *italic* are kept fixed).

- Models: Linear, 1/2/3 - Layer MLP
- Hidden dimension d: 10, 50, 100, 150, 200

- Equal Head: False
- Initialization: Kaiming/He

The results are reported in Figure 1. Four key observations emerge:

1. **Measured forgetting** consistently worsens with increasing architectural depth across all values of d.
2. The **Raw Proxy** G exhibits an initial $1/d$ scaling not attributable to averaging. For single-layer models, the decay is monotonic, whereas deeper models display slope inversion already at $d < 50$, deviating from measured forgetting.
3. The **Average Proxy** $\bar{G}$ provides a closer match to forgetting dynamics than its raw counterpart.
4. The **layer contribution** reveals opposite outcomes depending on whether we consider the averaged or raw proxy. While the contribution of deeper layers is lower in absolute value, it is higher in expectation, compared to the first layer.

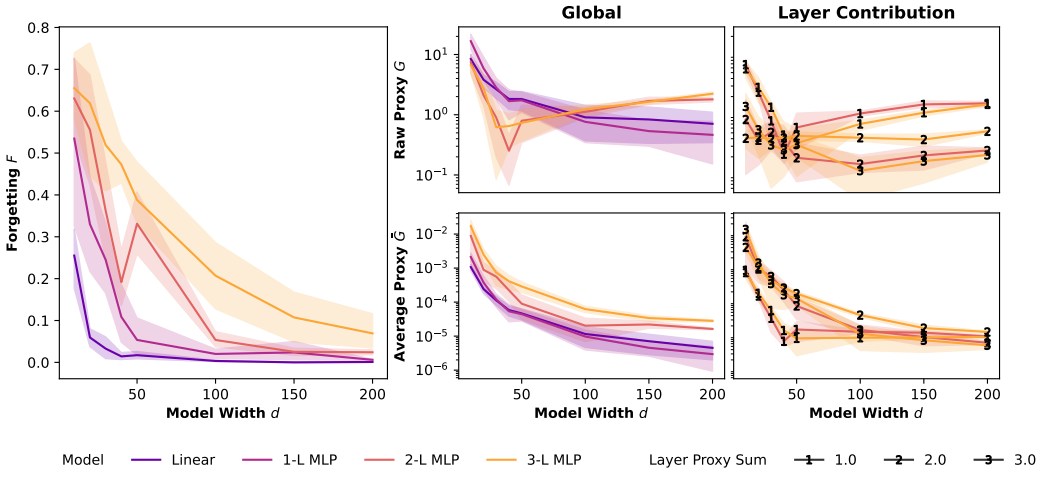


Figure 1: *Left*: measured forgetting for the four models as a function of d. *Center*: global proxy, raw (top) and averaged (bottom) versions. The global proxy is calculated by summing the proxies of different layers. *Right*: individual layer contributions to the proxy variant. The linear and 1-L MLP are not reported as they only have one layer. The y-scale of the proxies is logarithmic, and the one for forgetting is linear.

4.2 Equal Head Configuration

We observed in the previous experiment that the $1/d$ scaling does not come solely as an averaging of the proxy, since the raw G decreases for higher capacity values.

In the theory section, we hypothesized that scaling could arise either from cosine similarity between heads or from initialization-dependent head norms. To disentangle their individual contributions, we set the student task-specific heads equal to each other. In this way, cosine similarity has a unitary expectation and zero variance, isolating the effect uniquely from the initialization.

- Models: Linear, 1/2/3 - Layer MLP
- Hidden dimension d: 10, 50, 100, 150, 200
- Equal Head: True, False
- Initialization: Kaiming/He, Uniform $\mathcal{U}(-1, 1)$

The results, shown in Figure 2, reveal the following:

1. Enforcing **equal heads** cancels the $1/d$ scaling for both **forgetting** and the **raw proxy.**
2. **Initialization alone** is insufficient to explain the scaling effect.
3. The **averaged proxy** $\bar{G}$ continues to scale with $1/d$, as the normalization factor persists even when $\vartheta_t = \vartheta_{t'}$.
4. In deeper architectures, equalizing heads reduces the proxy's sensitivity. Moreover, the mismatch between proxy and forgetting increases with model depth, indicating more complex interference dynamics.

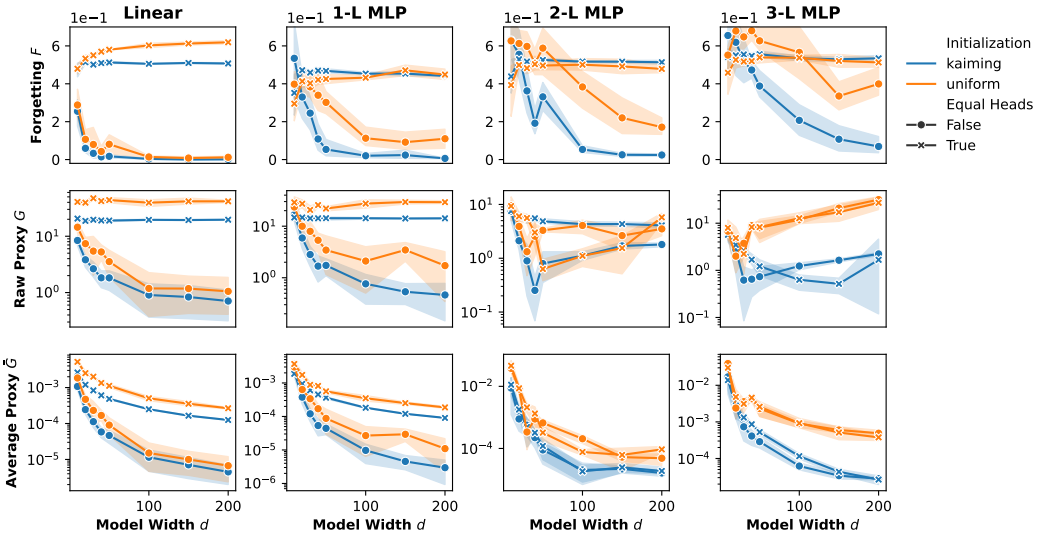


Figure 2: The rows divide the following metrics (from top to bottom): forgetting, raw proxy, and averaged proxy. Each column contains metrics for a given model (from left to right): Linear, 1-L MLP, 2-L MLP, and 3-L MLP. The y-scale of the proxies is logarithmic, and the one for forgetting is linear.

4.3 Task Similarity & Activations

Building on prior work that highlights the role of task similarity, we also examine its effect in the context of the $1/d$ scaling law. To this end, we conduct experiments on the Permuted MNIST dataset Goodfellow et al. (2015), where pixel positions are randomly permuted before training to systematically vary task similarity. In addition, we examine four activation functions (ReLU, GeLU Hendrycks & Gimpel (2023), ELU Clevert et al. (2016), and Leaky ReLU Maas et al. (2013)) to assess the robustness of the observed trends, as shown in Figure 3. The parameter **alpha** denotes the percentage of pixels that are randomly permuted. For instance, $\alpha = 0.8$ indicates that 80% of the pixels from the original image are permuted, thereby reducing task similarity.

5 Discussion

We observe that the necessary condition for observing the $1/d$ scaling is the orthogonality of the output heads and not the Kaiming/He initialization. This conclusion is supported by the equal-head setting, in which the cosine similarity has unit expectation and zero variance regardless of d. Under this condition, the $1/d$ scaling disappears, and the model resorts to residual memorization and random guessing of the previous task. Initializations can definitely accelerate scaling, as observed in the case of Kaiming/He initialization, but it is not the primary cause.

Furthermore, the Frobenius product between task gradients provides a reliable proxy for forgetting in shallow networks. However, as model complexity increases, discrepancies between the proxy and the measured forgetting become pronounced, thereby delineating the limits of the proxy's validity.

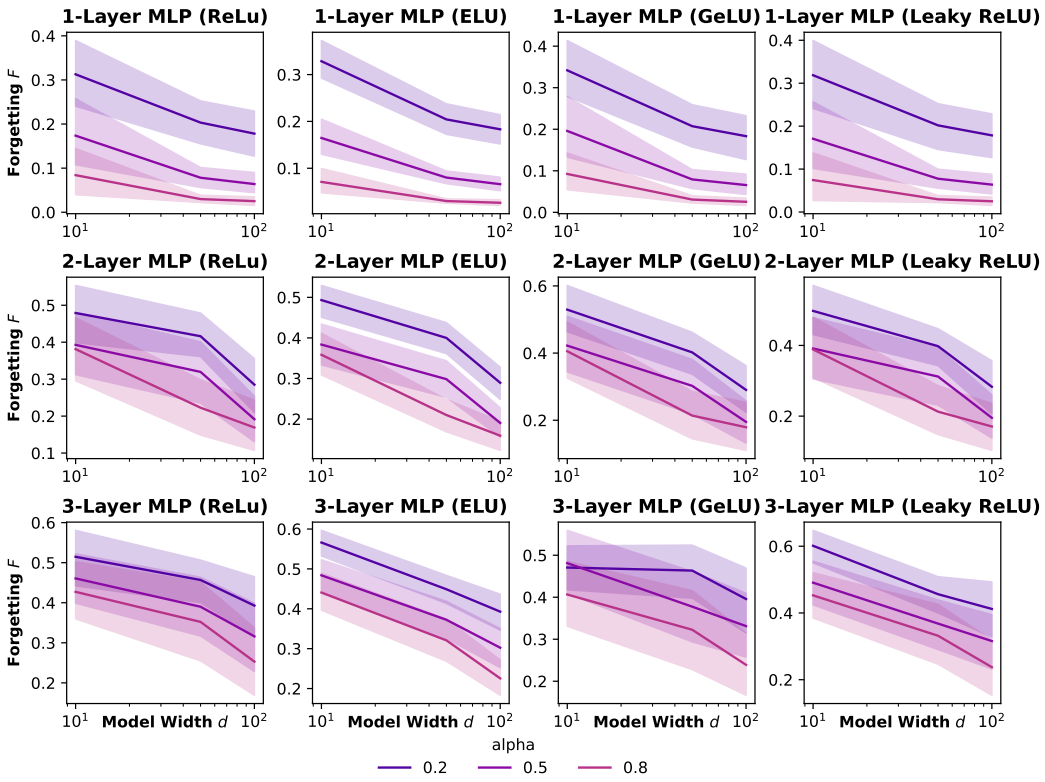


Figure 3: Relationship between forgetting and model width across different depths and activation functions.

6 Related Works

In this section, we review existing studies that analyze the influence of model width and depth on catastrophic forgetting.

6.1 On the Width

Several studies have reported a $1/d$ scaling law of forgetting, focusing primarily on its empirical advantages rather than its underlying mechanism. Mirzadeh et al. Mirzadeh et al. (2022) demonstrate the benefits of wider models by analyzing proxies such as gradient orthogonality and sparsity. They observe that wider networks exhibit greater inertia against drifting away from their initial minima when trained on new tasks, leading to improved memory retention. In addition, they report higher gradient sparsity for wider models when aggregating results across architectures. Building on their analysis, we link gradient behavior more directly to forgetting. We emphasize the limitations of a global proxy that covers both shallow and deep models, showing that deeper models deviate more substantially from the predictions of a simple gradient-based description.

A more recent study by Guha and Lakshman Guha & Lakshman (2024) introduces a theoretical framework based on overlapping rows per layer to connect forgetting with model width. Our findings are consistent with their description, since both head orthogonality and row overlap capture the degree of feature sharing between tasks in the feature extractor.

6.2 On the Depth

The inferior performance of deeper architectures has also been widely reported, although existing studies provide conflicting conclusions about which layers are most responsible for forgetting.

Even though Ramasesh et al. (2020) focuses mostly on task similarity, they also report the higher impact of deeper layers in forgetting. They used centered kernel alignment (CKA) Kornblith et al. (2019) between layers before and after having trained the model on the second task. They found lower similarity in deeper layers, concluding that **deep layers are mainly responsible for forgetting**. Our averaged version of the proxy leads to a similar conclusion, showing higher gradient conflicts in deeper layers, especially those closer to the output.

In contrast, Mirzadeh et al. Mirzadeh et al. (2022) argue that increasing depth does not yield the same monotonic gains in retention as increasing width. They establish a theoretical connection between forgetting and gradient norms, motivated by the intuition that larger parameter updates amplify interference from subsequent tasks. They report that the gradient norm of the first layer in an MLP increases with depth. Although they do not provide experimental evidence for deeper layers, they measure singular values larger than one, which inflates their theoretical upper bound for earlier layers compared to later ones. From this, they conclude that **earlier layers are the primary source of forgetting**. Their gradient-norm measure, $\|\partial_{\Theta_l}\mathcal{L}t'\|$, is proportional to the row-level proxy used in our work, but it does not capture gradient collisions between tasks. Under high task similarity, however, $G \approx \|\partial\Theta_l\mathcal{L}_{t'}\|^2$, and we recover their result that earlier layers in deep MLPs exhibit the largest absolute gradient norms.

We reconcile these seemingly contradictory results as follows. If forgetting is defined in terms of gradient collisions between tasks, then larger *absolute* collisions occur in earlier layers. At the same time, deeper units are, on average, more sensitive to distributional shifts induced by continual learning. Our analysis also shows that deeper layers are generally sparser than earlier ones, which accounts for their smaller absolute proxy values. The fact that, on average, deep units experience more interference can be explained by the non-linearity-induced alignment of the representations of the two tasks, but further investigation is needed, as this is beyond the scope of this study.

7 Conclusion

This study examined the origin of the $1/d$ scaling law in catastrophic forgetting and introduced a simple gradient-based proxy to isolate the key contributing factors. Although the proxy fails to fully capture forgetting in the most complex models, it offers a compact set of plausible quantities in which the $1/d$ scaling can arise.

We conclude that head orthogonality is the necessary condition for the scaling, reflecting the "curse of dimensionality" by which random vectors in high dimensions tend to be orthogonal. Initialization can accelerate the emergence of scaling, as in the case of Kaiming/He initialization, but it is not the root cause. Task similarity operates as a scaling factor of the proxy, with forgetting increasing proportionally to task similarity, as confirmed by our experiments. Finally, although the proxy was derived for ReLU activations, we observe that alternative activation functions such as GeLU, ELU, and Leaky ReLU exhibit qualitatively similar behavior.

References

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus), 2016. URL https://arxiv.org/abs/1511.07289.

Li Deng. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012. doi: 10.1109/MSP.2012.2211477.

Thang Doan, Mehdi Bennani, Bogdan Mazoure, Guillaume Rabusseau, and Pierre Alquier. A Theoretical Analysis of Catastrophic Forgetting through the NTK Overlap Matrix, February 2021.

William Falcon and The PyTorch Lightning team. PyTorch Lightning, March 2019. URL https://github.com/Lightning-AI/lightning.

Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. Orthogonal Gradient Descent for Continual Learning, October 2019.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization, 2021. URL https://arxiv.org/abs/2010.01412.

Ian J. Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks, 2015. URL https://arxiv.org/abs/1312.6211.

Etash Guha and Vihan Lakshman. On the Diminishing Returns of Width for Continual Learning, June 2024.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pp. 1026–1034, 2015.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023. URL https://arxiv.org/abs/1606.08415.

Naoki Hiratani. Disentangling and Mitigating the Impact of Task Similarity for Continual Learning, May 2024.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. doi: 10.1073/pnas.1611835114.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of Neural Network Representations Revisited, July 2019.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient Episodic Memory for Continual Learning, September 2022.

Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. Rectifier nonlinearities improve neural network acoustic models. In *Proc. icml*, volume 30, pp. 3. Atlanta, GA, 2013.

Michael McCloskey and Neal J. Cohen. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In Gordon H. Bower (ed.), *Psychology of Learning and Motivation*, volume 24, pp. 109–165. Academic Press, January 1989. doi: 10.1016/S0079-7421(08)60536-8.

Seyed Iman Mirzadeh, Arslan Chaudhry, Dong Yin, Huiyi Hu, Razvan Pascanu, Dilan Gorur, and Mehrdad Farajtabar. Wide Neural Networks Forget Less Catastrophically, July 2022.

Vinay V. Ramasesh, Ethan Dyer, and Maithra Raghu. Anatomy of Catastrophic Forgetting: Hidden Representations and Task Semantics, July 2020.

Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A Comprehensive Survey of Continual Learning: Theory, Method and Application, February 2024.

Jonathan Wenger, Felix Dangel, and Agustinus Kristiadi. On the Disconnect Between Theory and Practice of Neural Networks: Limits of the NTK Perspective, May 2024.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual Learning Through Synaptic Intelligence, June 2017.

A Appendix

A.1 Setup Details

A.1.1 Dataset and Task Definition

We conduct experiments on the MNIST dataset Deng (2012), which contains 28×28 grayscale images of handwritten digits (0–9). To study catastrophic forgetting within the teacher–student framework, the dataset is partitioned into different tasks:

- **Standard Split**: Task 1 consists of digits 0–4 and Task 2 of digits 5–9.
- **Variable Class Numbers**: The number of classes per task is controlled by $C \in \{2, 3, \ldots, 10\}$.
- **Permuted Tasks**: To vary task similarity, a fraction $\alpha \in [0, 1]$ of pixel positions is permuted. $\alpha = 0$ corresponds to no permutation and $\alpha = 1$ to a complete permutation. Both tasks contain the same classes, with Task 2 being a permuted version of Task 1.

Each image is flattened into a vector $X_t \in \mathbb{R}^{1\times m}$ with $m = 784$, and normalized to $[0, 1]$. Data is split into training (80%) and test (20%) sets using a fixed random seed.

A.1.2 Model Architectures

We evaluate four architectures:

1. **Linear Model**: A single linear transformation $h_t = X_t\Theta$ with $\Theta \in \mathbb{R}^{m\times d}$.
2. **One-/Two-/Three-Layer MLPs**: Stacks of linear layers with non-linear activation $\sigma(\cdot)$:

$$\begin{aligned} h_t &= \sigma(X_t\Theta_1) && \text{(1-Layer)} \\ h_t &= \sigma(\sigma(X_t\Theta_1)\Theta_2) && \text{(2-Layer)} \\ h_t &= \sigma(\sigma(\sigma(X_t\Theta_1)\Theta_2)\Theta_3) && \text{(3-Layer).} \end{aligned}$$

All models allow optional bias terms and configurable activations. Unless otherwise stated, we use ReLU activations without bias for analytical tractability.

A.1.3 Teacher-Student Framework

Our experimental setup follows a sequential teacher-student learning paradigm designed to study catastrophic forgetting:

We train two separate teacher models, each specialized on one task:

- **Teacher 1**: Trained exclusively on Task 1 data (e.g. digits 0-4)
- **Teacher 2**: Trained exclusively on Task 2 data (e.g. digits 5-9)

Both teachers use the same output dimensions corresponding to the number of classes.

The student model employs a dual-head architecture sharing a common feature extraction backbone. During training, only one head is active at a time, determined by the current task.

A.1.4 Initialization and Frozen Output Heads

Weight Initialization We employ multiple initialization strategies depending on the experimental focus:

- **Kaiming Initialization**: For ReLU networks, weights are initialized as:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{3}{n_{\text{in}}}}, \sqrt{\frac{3}{n_{\text{in}}}}\right) \tag{5}$$

- **Uniform Initialization**: For controlled initialization studies, weights are drawn from:

$$W_{ij} \sim \mathcal{U}(-a, a) \tag{6}$$

where $a \in \{0.1, 0.5, 1.0, 5.0\}$ is the uniform bound parameter.

All models (teachers and students) are initialized with the same random seed to ensure identical starting conditions. For student models, we investigate two head initialization strategies: identical initialization (`same_head = True`), where both output heads start with identical weights, and independent initialization (`same_head = False`).

Frozen Output Heads A critical aspect of our setup is that **all output heads are frozen during training**. Specifically:

- Teacher output layers ϑ_t are fixed.
- Student heads ϑ_1, ϑ_2 are fixed with `requires_grad` $=$ `False`.

Thus, task-specific adaptation occurs exclusively in the shared feature extractor Θ.

This constraint forces the networks to learn task-specific representations solely through modifications to the feature extraction layers, providing a controlled setting to study representation learning and interference.

A.1.5 Training Protocol

Optimization All models are trained using Stochastic Gradient Descent (SGD) with the following hyperparameters:

- Learning rate: $\eta = 0.01$
- Momentum: $\mu = 0.0$ (standard SGD)
- Batch size: 64
- Loss function: Cross-entropy loss

Training Schedule The training follows a sequential protocol:

1. **Teacher Training**: Each teacher is trained for 50 epochs on its respective task
2. **Student Training**: The student is trained sequentially:
   - 50 epochs on Task 1 (using Teacher 1's labels, head ϑ_1 active)
   - 50 epochs on Task 2 (using Teacher 2's labels, head ϑ_2)

During student training, the model learns to mimic the teacher's predictions rather than the ground truth labels, implementing a form of knowledge distillation in the sequential learning setting.

Hard/Soft Accuracy Forgetting F is measured as the accuracy drop between the first and second tasks (Equation 12).

The teacher-student setup allows you to choose the reference on which the accuracy is calculated.

1. **Hard** labels: the accuracy of the student is calculated with respect to the real labels of the data set.
2. **Soft** labels: the accuracy of the student is calculated with respect to the labels predicted by the teacher of the current task. This avoids the underfitting error at low capacity and provides a more precise forgetting measurement (see Figure 4).

To reduce the uncertainty coming from underfitting in low-capacity models, the soft accuracy was used in all our experiments to quantify forgetting.

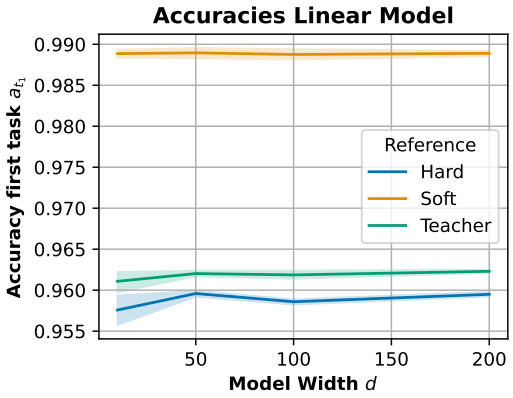


Figure 4: Comparison of accuracies in the first task calculated with hard and soft labels. The teacher's accuracy is also displayed. While the teacher and hard accuracies neck at low d, soft accuracies provide a stable baseline throughout the whole range of model widths.

A.1.6 Experimental Design

We conduct focused parameter sweeps to study multiple aspects of catastrophic forgetting:

- **Hidden dimensions**: $d \in \{10, 50, 100, 200\}$ for computational efficiency while covering key capacity regimes
- **Random seeds**: $\{1, 2, 3, 4, 5\}$ for statistical reliability
- **Number of classes per task**: $C \in \{2, 3, 4, 5, 6, 7, 8, 9, 10\}$ to study task complexity effects
- **Initialization bounds**: $a \in \{0.1, 0.5, 1.0, 5.0\}$ for uniform initialization studies
- **Student head initialization**: $\{\text{same}, \text{different}\}$ to study representation sharing
- **Task similarity**: $\alpha \in [0, 1]$ via input permutation parameter

This multi-dimensional parameter space enables systematic analysis of how model capacity, initialization strategy, task complexity, and task similarity interact to influence catastrophic forgetting phenomena.

Evaluation Metrics For each configuration, we track:

- Task-specific accuracy on validation sets after each training phase
- Cross-entropy loss for both heads throughout training
- Catastrophic forgetting metrics comparing performance before and after sequential training
- Gradient accumulation patterns and sharpness measures using SAM Foret et al. (2021)

All experiments are conducted using PyTorch Lightning Falcon & The PyTorch Lightning team (2019) with deterministic training enabled for reproducibility. The experimental framework supports both sequential execution and parallel execution via SLURM job arrays for efficient large-scale parameter sweeps.

A.2 Complementary Experiments

A.2.1 Number of Classes

We first investigate the influence of the number of classes on forgetting in the equal-head setting. This experiment aims to verify that the number of classes modulates the d-independent plateau, which emerges due to random guessing of the "forgotten" labels. To fully utilize the ten classes of MNIST, both tasks include all digits from 0 to 9. The second task is obtained by permuting the first task with a permutation coefficient of $\alpha = 0.8$. The experimental configuration is summarized below:

- Models: Linear
- Hidden dimension d: 10,200
- Equal Head: True
- Initialization: Kaiming/He

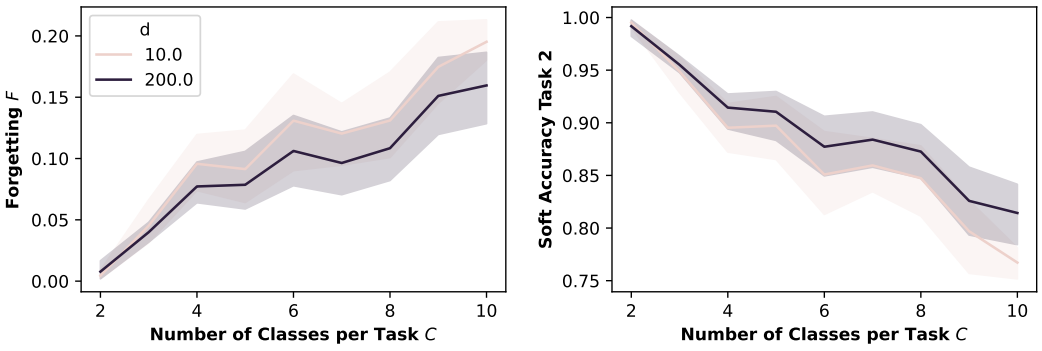


Figure 5: Forgetting as a function of the number of classes in the equal-head setting.

The results in Figure 5 show that forgetting increases with the number of classes, reflecting the expected drop in accuracy after training on the second task. Interestingly, the degradation progresses more slowly than $1/C$, which would correspond to purely random guessing. This indicates that random guessing explains only part of the behavior, while a substantial fraction of accuracy is preserved through partial recall of the first task.

A.2.2 Bounds of Uniform Initialization

We next analyze the effect of varying the d-independent bound a in uniform initialization, in order to isolate the role of initialization on forgetting and its relationship to our gradient-based proxy. The setup is as follows:

- Models: Linear, 1/2/3-Layer ReLU MLP
- Hidden dimension d: 10, 50, 100, 200
- Equal Head: True/False
- Initialization: Uniform $\mathcal{U}(-a, a)$

The results in Figure 6 demonstrate that forgetting degrades proportionally to the magnitude of a. As previously observed in Figure 2, proxies for deeper models deviate substantially from the measured forgetting. Notably, while setting equal heads effectively suppresses the $1/d$ scaling of forgetting, this choice has little impact on the proxies. Instead, the proxies are predominantly driven by the initialization bound a, highlighting its central role in shaping their dynamics.

A.3 Formal Definitions

This appendix derives the forgetting proxy and related identities using linear algebra under the column-vector convention.

List of Symbols

n_t : number of samples of task t
$\mathbf{X}_t$: task batch $\in \mathbb{R}^{n_t \times m}$
X_t : task sample $\in \mathbb{R}^{1 \times m}$
d_l : hidden dimension of layer l
o : output dimension (number of classes in a classification task)

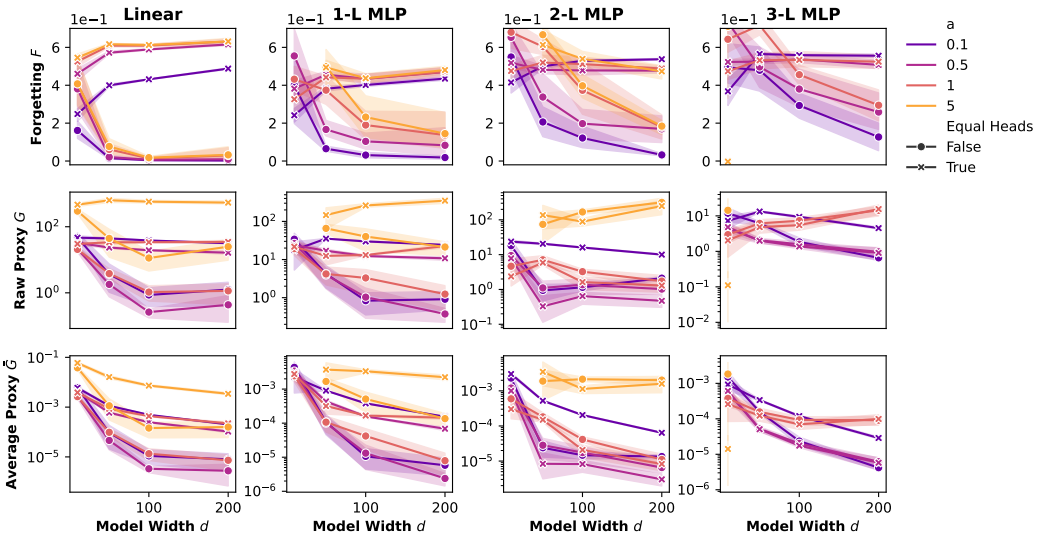


Figure 6: Effect of the uniform initialization bound a on forgetting and the proxy. Certain points are omitted because SGD failed to converge for combinations of large a and small d.

σ : ReLU activation function, $\sigma(z) := \max(0, z)$
Θ : hidden layer $\in \mathbb{R}^{m \times d}$
ϑ_t : output head (for task t) $\in \mathbb{R}^{d \times o}$
z_t : pre-activations $z_t = X_t\Theta \in \mathbb{R}^{1 \times d}$
h_t : activations $h_t = \sigma(z_t) \in \mathbb{R}^{1 \times d}$
E_t : residuals $E := f(X_t) - Y_t \in \mathbb{R}^{1 \times o}$
δ_t : logit residuals $\delta_t := \text{Softmax}(f(X_t)) - Y_t \in \mathbb{R}^{1 \times o}$
L_t : loss task t
$\mathcal{L}_t^{(i)}$: i^{th} sample contribution to loss task t
t : Task index, we often use t' to indicate the task $\neq t$, usually following t
I_d : d-dimensional identity

Mathematical Identities

Here, we collect all the mathematical identities that were used to derive the results of the following parts.

Frobenius product rank one identity

$$\begin{aligned} \langle AB^\top, CD^\top \rangle_F &= \text{Tr}(B^\top A^\top CD) \\ &= (A^\top C)(B^\top D) \\ &= \langle A, C \rangle_F \langle B, D \rangle_F \end{aligned} \tag{7}$$

Vec and Unvec
Vectorizing a matrix basically means to flatten it along one dimension.

$$\Theta \in \mathbb{R}^{m \times d} \to \theta := \text{vec}(\Theta) \in \mathbb{R}^{1 \times md} \tag{8}$$

The original matrix is retrieved by broadcasting the vectorized matrix back using the unvec operation.

Kronecker Product
$\otimes_{\text{Kr}}$ denotes the Kronecker product of two matrices.

$$(k \times j) \otimes_{\text{Kr}} (m \times n) \to (km \times jn) \tag{9}$$

Vec & Kronecker Product

$$\underbrace{a}_{1\times n} \otimes_{\text{Kr}} \underbrace{b}_{1\times m} = \underbrace{\text{vec}(b^\top a)}_{1\times(nm)} \tag{10}$$

The *Vec Trick*

$$(A \otimes_{\text{Kr}} B)\,\text{vec}(C) = \text{vec}(BCA^\top) \tag{11}$$

Forgetting Given a model f in a classification task, forgetting F is measured as the accuracy A_f drop between the first t and second tasks t'.

$$F(f, X_t, X_{t'}) := A_f(t) - A_f(t') \tag{12}$$

Proxy We use the Frobenius product between the gradients of task t and task t' as a proxy for forgetting:

$$G := \langle \nabla_\Theta \mathcal{L}_t, \nabla\Theta\mathcal{L}_{t'} \rangle_F. \tag{13}$$

Loss Function Considering n_t samples for task t, the training loss is defined as the mean training loss of the sample.

$$\text{L}_t = \frac{1}{n_t}\sum_{i=1}^{n_t} \mathcal{L}_t^{(i)} \tag{14}$$

Depending on the tasks, a common choice for regression is the **Mean Squared Error**.

$$\mathcal{L}_t^{(i)} = \frac{1}{2}\|E_t^{(i)}\|^2 = \frac{1}{2}(f(X_t^{(i)}) - Y_t^{(i)})(f(X_t^{(i)}) - Y_t^{(i)})^\top \tag{15}$$

While for multi-class classification is the **Cross-Entropy**

$$\mathcal{L}_t^{(i)} = -\sum_{c=1}^{o} Y_{t,c}^{(i)} \log\Big(\text{Softmax}\big(f_t(X_t^{(i)})\big)\Big) \tag{16}$$

Applying the chain rule on a generic loss $\mathcal{L}$, the vectorized gradient with respect to the layer parameter Θ can be separated into a loss-dependent and model-dependent component.

$$\nabla_\Theta \mathcal{L} = \underbrace{\nabla_f \mathcal{L}}_{\text{loss}^\dagger} \cdot \underbrace{J_f(\Theta)}_{\text{model}^\ddagger} \tag{17}$$

† since $\mathcal{L}$ is a scalar-valued function, this term is the gradient w.r.t. the model function f, $\nabla_\Theta \mathcal{L} \in \mathbb{R}^{1\times o}$

‡ is the Jacobian of the model output w.r.t the parameter $\Theta \in \mathbb{R}^{m\times d}$. The model f is generally a vector-valued function if $o > 1$. Therefore, it is a Jacobian rather than a gradient and $J_f(\Theta) \in \mathbb{R}^{o\times m\times d}$

In the vectorized form $\theta = \text{vec}(\Theta)$

$$\nabla_\theta \mathcal{L} = \underbrace{\nabla_f \mathcal{L}}_{1\times o} \cdot \underbrace{J_f(\theta)}_{o\times md} \tag{18}$$

The term † can be calculated independently of the model considered.

$$k := \nabla_f \mathcal{L} = \begin{cases} E_t := f_t(X_t^{(i)}) - Y_t^{(i)} & \text{MSE} \\ \delta_t := \text{Softmax}\big(f_t(X_t^{(i)})\big) - Y_t^{(i)} & \text{Cross-Entropy} \end{cases}$$

In the next derivations, E_t and δ_t can be interchanged depending on the loss function.

Without loss of generality, we set $o = 1$ and e. The Jacobian becomes real valued and can be substituted by the gradient $\nabla_\theta f$

$$\begin{aligned} \nabla_\theta \mathcal{L} &= k \nabla_\theta f && \text{flattened} \\ \nabla_\Theta \mathcal{L} &= k \nabla_\Theta f && \text{original shape} \end{aligned} \tag{19}$$

The rest of the appendix deals with the calculation of $\nabla_\theta f$ for different models.

A.3.1 Linear Model

$$\begin{aligned} f(X) &= z\vartheta \\ &= X\Theta\vartheta \end{aligned}$$

The gradient for Equation 19 is decomposed by the chain rule.

$$\nabla_\theta f = \nabla_z f \cdot J_z(\theta)$$

$$\begin{aligned} \nabla_z f &= \vartheta^\top \\ J_z(\theta) &= (I_d \otimes_{\text{Kr}} X) \end{aligned}$$

Substituting into the chain, the flattened gradient is obtained.

$$\begin{aligned} \nabla_\theta f &= \vartheta^\top \otimes_{\text{Kr}} X \\ &= \text{vec}(X^\top \vartheta^\top) \end{aligned}$$

Broadcasting the gradient back to its original shape by the unvec, one gets

$$\nabla_\Theta f = \text{unvec}(\nabla_\theta f) = X^\top \vartheta^\top \tag{20}$$

Substitute in Equation 19,

$$\begin{aligned} \nabla_\Theta \mathcal{L}_t &= e X^\top \vartheta^\top \\ &= X^\top e \vartheta^\top \end{aligned}$$

It is now possible to calculate G. For this, it is necessary to introduce the subscripts identifying different tasks t, t' into the notation.

$$\begin{aligned} G &= \langle \nabla_\Theta \mathcal{L}_t, \nabla_\Theta \mathcal{L}_{t'} \rangle_F \\ &= \langle X_t^\top e_t \vartheta_t^\top, X_{t'}^\top e_{t'} \vartheta_{t'}^\top \rangle_F \\ &= \underbrace{k_t k_{t'}}_{K} \langle X_t^\top \vartheta_t^\top, X_{t'}^\top \vartheta_{t'}^\top \rangle_F \\ &= K \langle X_t, X_{t'} \rangle \langle \vartheta_t, \vartheta_{t'} \rangle && \text{by identity 7} \end{aligned}$$

In expectation, G is zero since $\vartheta_t \overset{\text{iid}}{\sim} U(-a, a)$ - but the variance is non-zero.

A.3.2 ONE-LAYER MLP

The mapping f can be written equivalently for the task batch $\mathbf{X}_t$ or for a single sample X_t.

$$\begin{aligned} f(X_t) &= h\vartheta_t \\ &= \sigma(z_t)\vartheta_t \\ &= \sigma(X_t\Theta)\vartheta_t \end{aligned}$$

The Jacobian ‡ for Equation 19 can be decomposed by applying the chain rule and the vectorized parameter $\theta = \mathrm{vec}(\Theta)$

$$\nabla_\theta f = \underbrace{\nabla_h f}_{\in \mathbb{R}^{1\times d}} \cdot \underbrace{J_h(z)}_{\in \mathbb{R}^{d\times d}} \cdot \underbrace{J_z(\theta)}_{\in \mathbb{R}^{d\times md}} \tag{21}$$

Each component is calculated as follows.

$$\begin{aligned} J_f(h) &= \vartheta_t^\top \\ J_h(z) &= D_t \\ J_z(\theta) &= I_d \otimes_{\mathrm{Kr}} X_t \end{aligned} \tag{22}$$

Plugging 22 into 21

$$\begin{aligned} \nabla_\theta f &= (\vartheta_t^\top D_t) \otimes_{\mathrm{Kr}} X_t \\ &= \mathrm{vec}(X_t^\top (\vartheta_t^\top D_t)) && \text{by 10} \end{aligned}$$

By the unvec operation

$$\nabla_\Theta f = \mathrm{unvec}(\nabla_\theta f) = X_t^\top \vartheta_t^\top D_t$$

Substitute in 19

$$\nabla_\Theta \mathcal{L}_t = k_t X_t^\top \vartheta_t^\top D_t$$

We can now write G

$$\begin{aligned} G &= \langle \nabla_\Theta \mathcal{L}_t, \nabla_\Theta \mathcal{L}_{t'} \rangle_F \\ &= k_t k_{t'} \langle X_t^\top \vartheta_t^\top D_t D_{t'}^\top \vartheta_{t'} X_{t'} \rangle_F && (23) \\ &= K\langle X_t, X_{t'} \rangle \cdot \mathrm{Tr}(\vartheta_t^\top D_t D_{t'}^\top \vartheta_{t'}) && \text{by 7} \quad (24) \end{aligned}$$